Jenkins Miscellaneous

starts at 9:05 pm

Agenda

1. Reports
2. Error Handling
3. Retry Mechanism
4. Jenkins Administration
    1. Upgrades
    2. Backups and Restore
5. Jenkins Monitoring
6. Shared Libraries

→ Reports

They allow developers and teams to visualize the status of their builds, tests, and code quality metrics.

(1) Test Results
(2) Code coverage
(3) Build health

→ Junit (unit testing in Java)

Plugins
→ junit
→ surefire

```
pipeline {
  agent any
  stages {
    stage('Test') {
      steps {
        sh 'mvn test'  // Running tests
        junit 'target/surefire-reports/*.xml'  // Publish JUnit reports
      }
    }
  }
}
```

Maven
→ simplifies Build
→ manages dependencies
→ ensures consistency

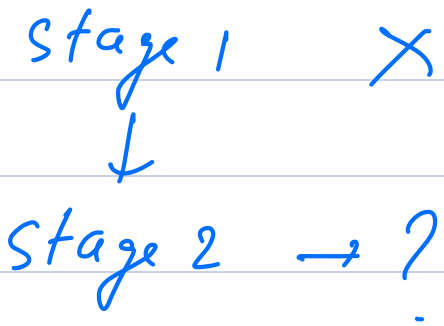
mvn test ?

1. **Compiles** the source code (`mvn compile`).
2. **Compiles** the test code (`mvn test-compile`).
3. **Executes** all test cases in the `src/test/java` directory.

/mvn
→ source
→ test

4. **Generates** a test report (`target/surefire-reports`).

→ Error handling



Error handling in Jenkins pipelines ensures that failures are **gracefully managed** instead of causing the entire pipeline to crash unexpectedly.

currentBuild.result = 'FAILURE'

```
pipeline {
    agent any

    stages {
        stage('Create File') {
            steps {
                script {
```

```
            sh 'echo Hello Jenkins > /tmp/demo_file.txt'
            echo "File created successfully!"
        }
    }
}

stage('Modify File (Simulated Failure)') {
    steps {
        script {
            try {
                sh 'some_invalid_command' // This will fail
                echo "File modified successfully!"
            } catch (Exception e) {
                echo "Error in modifying file: ${e.getMessage()}"
                currentBuild.result = 'FAILURE' // Explicitly fail the pipeline
            }
        }
    }
}

stage('Delete File') {
    steps {
```

```
            script {
                sh 'rm /tmp/demo_file.txt'
                echo "File deleted successfully!"
            }
        }
    }
}

post {
    always {
        echo "Pipeline execution completed!"
        echo "Performing cleanup tasks..."
        sh 'rm -f /tmp/demo_file.txt'
    }
    success {
        echo " All stages executed successfully!"
    }
    failure {
        echo " Pipeline failed due to an error!"
    }
}
}
```

Retry Mechanism

```
retry(3) {
    sh 'some_command'
}

pipeline {
    agent any

    stages {
        stage('Download File') {
            steps {
                script {
                    retry(3) {
                        sh 'curl -o /tmp/sample.txt https://example.com/sample.txt'
                    }
                }
            }
        }

        stage('Process File') {
```

```
        steps {
            script {
                if (fileExists('/tmp/sample.txt')) {
                    echo 'File downloaded successfully, proceeding with processing...'
                } else {
                    error 'File not found after retries!'
                }
            }
        }
    }
}

post {
    always {
        echo 'Cleaning up workspace...'
        sh 'rm -f /tmp/sample.txt'
    }
}
```

→ Jenkins Administration

→ Upgrades

1. Go to **Manage Jenkins** and check for Jenkins & security alerts. If an upgrade is available, it will show at the top of the Manage Jenkins page.

2. Right-click on **changelog** in the alert panel to open the changelog for this release in a new window or tab. Read this to be aware of changes this upgrade could make to your system.

3. Click **download** to download the upgrade software.

4. jenkins --version

2.497

6. Replace the `jenkins.war` file on your system with the new file.

    1. ls -lrt *.war

    2. scp -i "scaler.pem" jenkins.war ubuntu@16.16.197.45:/tmp/

7. On Linux systems, this is located in the `/usr/share/jenkins` directory by default.

    1. sudo find . -name '*jenkins.war*'

    2. Replace in usr/share/java

↳ java ☆

8. Restart Jenkins to apply the upgrade. (systemctl restart jenkins)

9. jenkins --version

→ Backup and Restore

→ Why backups?

① D.R.

→ Recover some old configuration.

→ Methods

① Filesystem snapshot.

② Plugins for backup.

→ thinbackup

Break → 10:20 pm

Moving Backup to remote server.

/var/lib/jenkins/thinbackup.

↓ directory.

rsync ?

To sync the backup to a remote server (e.g., using SCP):

scp -r /var/lib/jenkins/thinbackup/ user@remote-server:/backup/

Or using **rsync** (for incremental transfer):

rsync -avz /var/lib/jenkins/thinbackup/ user@remote-server:/backup/

/abcd

1. txt
2. "
3. txt

/abcd

1. txt
2. txt

Feature	`scp`	`rsync`
Transfer Mode	Copies files from source to destination	Synchronizes files and only transfers changes
Speed	Slower (copies everything)	Faster (only transfers differences)
Resume Support	No (must restart transfer)	Yes (`--partial` allows resuming)
Compression	No	Yes (`-z` enables compression)
Directory Sync	No (recursively copies directories but no sync)	Yes (keeps source and destination in sync)
Bandwidth Efficient	No (copies entire files)	Yes (transfers only changed parts of files)
SSH Support	Yes (built-in)	Yes (uses SSH by default)
Deletion Handling	No	Yes (`--delete` removes deleted files on destination)
Preserves Permissions & Timestamps	Yes (`-p` flag)	Yes (`-a` flag)

→ plugins, binaries, dependencies

(3) → Shell script for Backup:

→ /var/lib/jenkins

```
#!/bin/bash

backup_dir="/backup/jenkins"

mkdir -p $backup_dir

tar -czvf $backup_dir/jenkins_backup_$(date +%F).tar.gz /var/lib/jenkins/
```

crontab -e

0 2 * * * /script.sh

→ Manual Restore from a Backup.

stop Jenkins

systemctl stop jenkins

Move Existing Data

mv /var/lib/jenkins /var/lib/jenkins_old

Restore the Backup

cp -r /backup/jenkins/* /var/lib/jenkins/

Set Correct Permissions

chown -R jenkins:jenkins /var/lib/jenkins

Restart Jenkins

systemctl start jenkins

→ Jenkins Monitoring

→ Why ?

① ensure server is healthy
② understand how resources are getting utilised
③ identify issues

① Logs
↓
system logs.

② Nodes.

③ Monitor executors.
↓
Load statistics.

④ Monitoring Plugin

monitoring → Plugin name

→ Shared libraries

A **Jenkins Shared Library** is a reusable, version-controlled set of Groovy scripts that can be used across multiple Jenkins pipelines.

Key Benefits:

Reusability – Write pipeline functions once and use them in multiple projects.

Maintainability – Easier updates since logic is stored in a single place.

Modularity – Organize common pipeline steps into separate, reusable methods.

Version Control – Store in a Git repository for tracking and rollback.

→ Demo

```
testing_repo/
├── vars/           # Global functions
|   ├── printMessage.groovy  # Example function
├── README.md       # Documentation
```